

Calcolatori Elettronici

10 febbraio 2025

Teoria

Esercizio 1 (7 punti): Siano $A = -4X$ e $B = 7Y$ due numeri interi, in cui X e Y sono, rispettivamente, il penultimo ed ultimo numero della propria matricola—occorre *sostituire* le cifre della matricola a X e Y , non eseguire una moltiplicazione. Calcolare la rappresentazione binaria, in complemento a due a 8 bit, di tali valori. Eseguire l'operazione di somma tra i due numeri nella loro rappresentazione binaria, mostrando tutti i passaggi. Specificare se si verifica un overflow. Convertire il risultato finale in decimale e verificare la correttezza dell'operazione.

Esercizio 2 (10 punti): Si realizzi una rete sequenziale in grado di determinare se, in una sequenza infinita di caratteri appartenenti all'alfabeto $I = \{a, b\}$, all'interno di gruppi di 3 caratteri consecutivi sono presenti un numero di a pari. Per ciascun gruppo di 3 caratteri, la rete restituirà come output il valore `pari` o `dispari`. Fino a che non è stato ricevuto un gruppo di 3 caratteri, l'uscita assumerà il valore `?`. Si realizzi il circuito equivalente dell'equazione di eccitazione minimizzata di un flip flop di stato a scelta.

Esercizio 3 (8 punti): Disegnare l'architettura interna del processore z64. Descrivere il microcodice atto a supportare l'esecuzione dell'istruzione `jc .target`, compresa la fase di fetch.

Esercizio 4 (5 punti): Discutere le differenti modalità di accesso al BUS di sistema da parte di un DMAC.

Soluzioni

Esercizio 1

Si suppongono $X=5$ e $Y=6$. Per determinare la rappresentazione di -45 , convertiamo prima il numero in base 2, con divisioni successive. Si ottiene $45_{10} = 00101101_2$. Per calcolare il complemento a due, calcoliamo prima il complemento a uno invertendo tutti i bit, ottenendo: $00101101_2 \rightarrow 11010010_2$. A questo punto sommiamo 1: $11010010_2 + 1 = 11010011_2$, ottenendo che $-45_{10} = 11010011_2$

Per 76 possiamo direttamente convertire il valore in binario con divisioni successive, ottenendo $76_{10} = 01001100_2$.

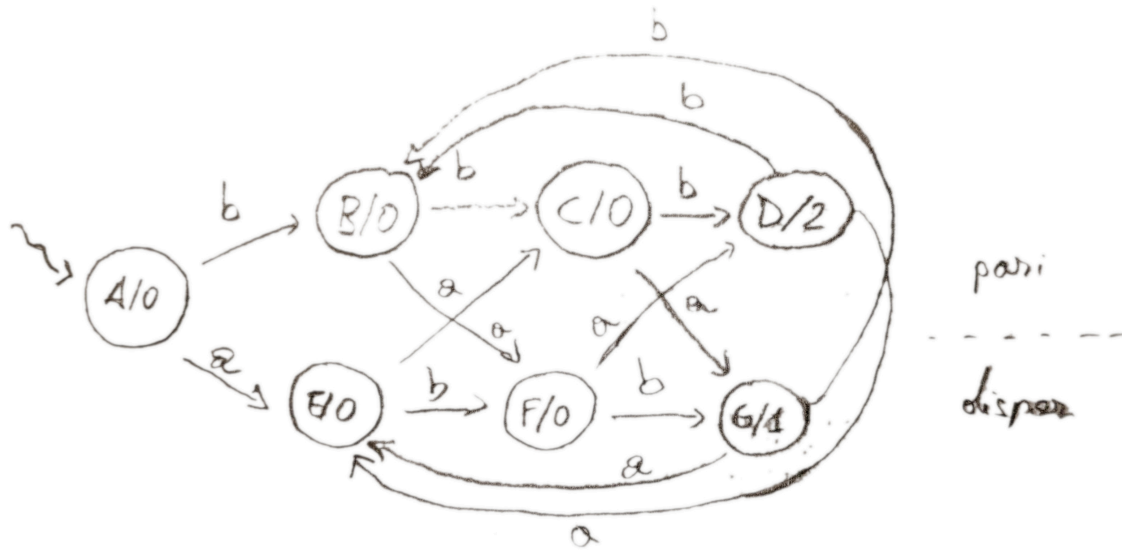
La somma si ottiene effettuando la somma bit a bit:

$$\begin{array}{r} 1\ 1\ 0\ 1\ 0\ 0\ 1\ 1 \\ 0\ 1\ 0\ 0\ 1\ 1\ 0\ 0 \\ \hline 0\ 0\ 0\ 1\ 1\ 1\ 1\ 1 \end{array}$$

con ultimo riporto pari a 1 e penultimo riporto pari a 1. Poiché gli ultimi due riporti sono concordi, non vi è overflow. Ciò era prevedibile poiché, essendo gli operandi di segno opposto, il risultato è sempre rappresentabile. Infatti: $00011111_2 = 31_{10} = -45 + 76$.

Esercizio 2

L'automa per riconoscere il numero di a in gruppi di 3 caratteri può essere il seguente (macchina di Moore):



con le seguenti codifiche:

$$I = \{a, b\}$$

$$O = \{?, \text{pari}, \text{dispari}\}$$

$$S = \{A, B, C, D, E, F, G\}$$

i	x
a	0
b	1

σ	$z_1 z_0$
?	00
dispari	01
pari	10

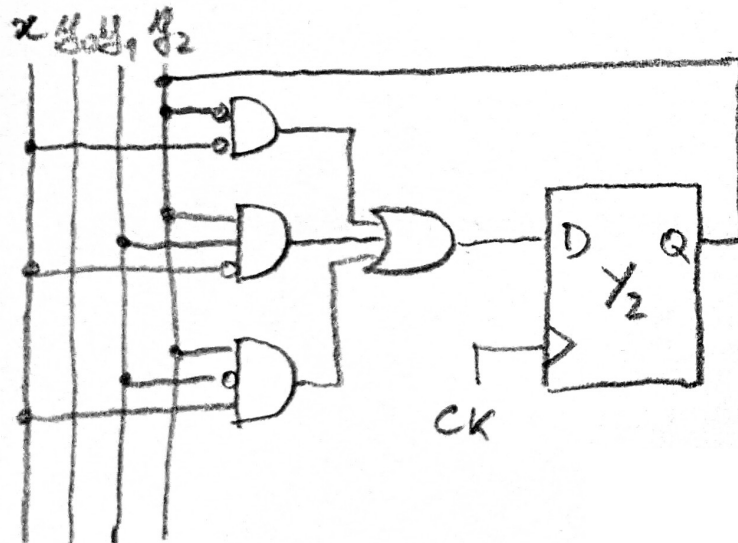
λ	y_2	y_1	y_0
A	0	0	0
B	0	0	1
C	0	1	0
D	0	1	1
E	1	0	0
F	1	0	1
G	1	1	0

Si può quindi definire la seguente tabella degli stati e transizioni, scegliendo di minimizzare l'equazione di y'_2 :

$y_2 y_1 y_0 x$	$y_2' y_1' y_0'$
0 0 0 0	1 0 0
0 0 0 1	0 0 1
0 0 1 0	1 0 1
0 0 1 1	0 1 0
0 1 0 0	1 1 0
0 1 0 1	0 1 1
0 1 1 0	1 0 0
0 1 1 1	0 0 1
1 0 0 0	0 1 0
1 0 0 1	1 0 1
1 0 1 0	0 1 1
1 0 1 1	1 1 0
1 1 0 0	1 0 0
1 1 0 1	0 0 1

$y_2 y_1$	$y_0 x$	00	01	11	10
00		1	1	1	0
01		0	0	0	1
11		0	0	-	1
10		1	1	-	0

Portando quindi all'equazione di eccitazione $y_2' = \overline{y_2 x} + y_2 y_1 \overline{x} + y_2 \overline{y_1} x$ che può essere realizzata con il circuito:



Esercizio 3

```

1 MAR ← RIP
2 MDR ← (MAR); RIP ← RIP + 8
3
4
5 IR ← MDR
6 IF FLAGS[CF] = 1 THEN
7     TEMP1 ← RIP
8     TEMP2 ← IR[0:31]
9     RIP ← ALU OUT[ADD]
10 ENDIF

```

```

BRIP = 1, WMAR = 1
INCRIP = 1, BAB = 1
BAB = 1, MRD = 1
BAB = 1, MRD = 1, BDBIN = 1, WMDR = 1
WIR = 1, BMDROUT = 1

BRIP = 1, WT1 = 1
BSHORT = 1, WT2 = 1
Aopcode = 0000, BA = 1, WRIP = 1

```

Progetto

Un processore `z64` controlla un sistema di verifica di codici a barre, costituito da una periferica `LETTORE` ed una periferica `LUCE`. La periferica `LETTORE` utilizza uno scanner ottico per acquisire il valore numerico codificato in un codice a barre. Tale valore, di dimensione longword, utilizza il bit meno significativo come bit di parità pari.

`LETTORE` opera in modalità asincrona. Alla pressione del pulsante di lettura del codice a barre, `LETTORE` avvisa il processore `z64` il quale recupera da un apposito registro di interfaccia il valore acquisito. Nel caso in cui non sia stato possibile acquisire alcun valore (ad esempio, perché il lettore ottico non stava inquadrando alcun codice a barre), il valore comunicato allo `z64` corrisponde a -1.

In caso di lettura valida, il processore deve verificare se il codice a barre è corretto—ossia, se il bit di parità è coerente con i restanti bit memorizzati nella longword. Qualora il valore non sia corretto, il codice viene memorizzato in un array `codici_errati`, di dimensione 1024 elementi. L'array viene utilizzato come buffer circolare: quando è pieno, si riprende a scrivere nel primo elemento, sovrascrivendo eventuali elementi precedentemente memorizzati.

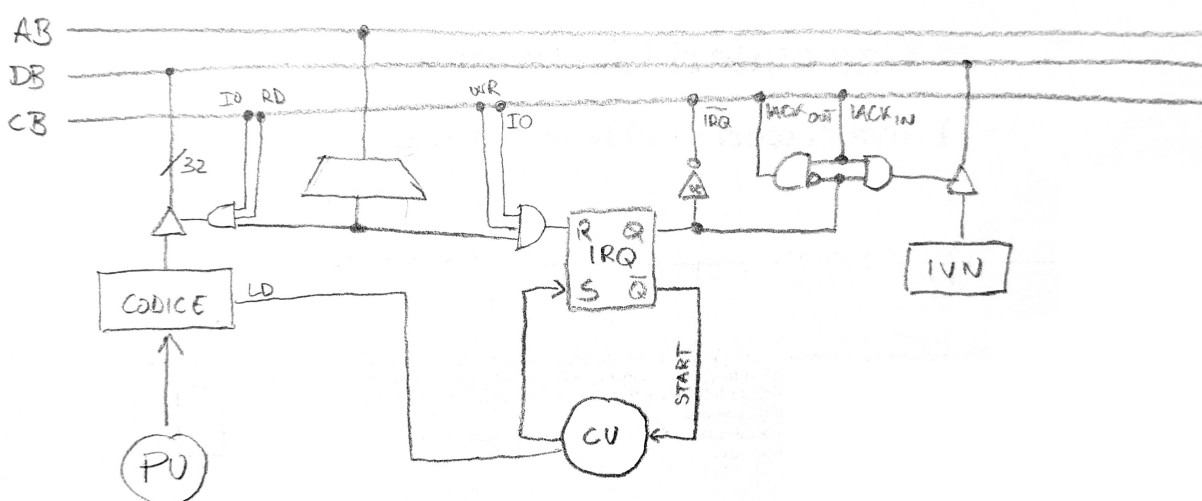
Al termine dell'operazione di verifica, lo `z64` utilizza la periferica `LUCE` (sincrona) per comunicare all'utente il risultato dell'operazione. `LUCE` utilizza un led che può essere acceso con 3 differenti colori: nel caso in cui non sia stato possibile acquisire un valore, la luce dovrà assumere il colore rosso; nel caso in cui il codice acquisito non superi la verifica di parità, la luce dovrà assumere il colore giallo; in caso di codice corretto, la luce dovrà assumere il colore verde.

Realizzare:

- Le interfacce dei dispositivi `LETTORE` e `LUCE`;
- Tutto codice necessario al funzionamento del sistema, senza l'utilizzo di pseudo-istruzioni.

Soluzione

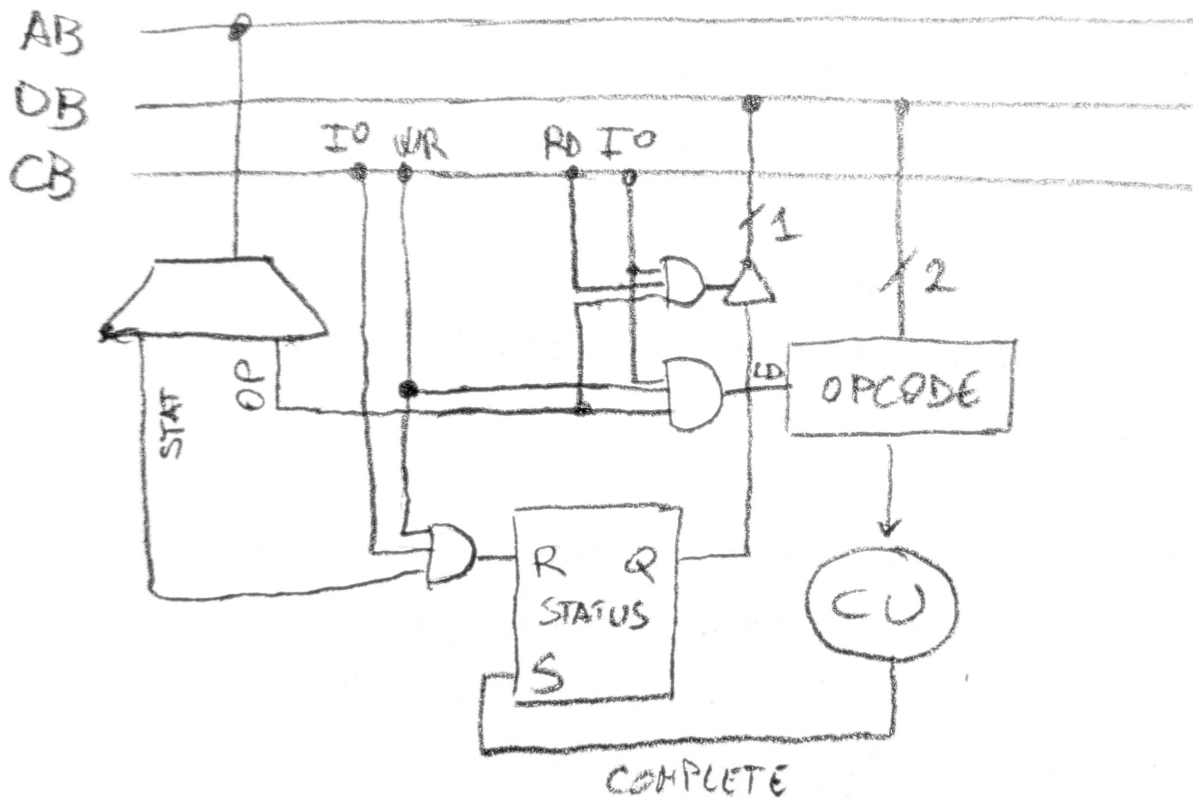
L'interfaccia di `LETTORE` può essere realizzata come segue:



trattandosi di una periferica asincrona di input. È possibile utilizzare l'output $\overline{Q}$ di IRQ come segnale di start poiché la periferica è attivata dalla pressione del pulsante ed invia quindi richieste di interruzione autonomamente, ma occorre evitare che si sovrascriva il valore nel registro codice (ad esempio con una seconda pressione del pulsante) prima che il processore abbia processato il valore precedente. Ciò può essere comunicato al termine del

driver con la cancellazione della causa di interruzione.

LUCE è una periferica sincrona che necessita di un opcode per indicare quale colore deve assumere la luce:



Una possibile implementazione del software è la seguente.

```

1  .org 0x800
2  .data
3      .equ LETTORE, 0x0
4      .equ LUCE_OP, 0x1
5      .equ LUCE_STAT, 0x2
6      .equ BUF_SIZE 1024
7      codici_errati: .fill BUF_SIZE, 4
8      pos: .word 0
9      .equ ROSSO, 0 # opcode per LUCE
10     .equ GIALLO, 1
11     .equ VERDE, 2
12 .text
13 main:
14     sti
15     .loop:
16         hlt
17         jmp .loop
18
19 # Restituisce `true` se la parità è corretta
20 check_parity:
21     cml $0, %edi
22     jp .ok
23     movq $0, %rax
24     ret
25 .ok:
26     movq $1, %rax
27     ret
28
29 # Memorizza un valore nel prossimo slot di codici_errati
30 store:

```

```

31     movzwb pos, %rcx
32     movl %edi, codici_errati(, %rcx, 4)
33     addq $1, %rcx
34     andq $BUF_SIZE-1, %rcx
35     movw %cx, pos
36     ret
37
38 .driver 0 # LETTORE
39     push %rax
40     push %rdi
41     push %rcx
42     inl $LEETTORE, %eax # Leggi il codice
43     movl %eax, %edi
44     movb $ROSSO, %r12b
45     cmpl $-1, %eax # Se -1, accendi colore rosso
46     jz .accendi
47     movb $GIALLO, %r12b
48     call check_parity
49     testq %rax, %rax # false: scrivi nel vettore e accendi colore giallo
50     jz .copia
51     movb $VERDE, %r12b
52     jmp .accendi # controlli superati: accendi colore verde
53 .copia:
54     call store
55 .accendi:
56     movb %r12b, %al
57     outb %al, $LUCE_OP
58     outb %al, $LUCE_STAT
59 .bw:
60     inb $LUCE_STAT, %al
61     btb $0, %al
62     jnc .bw
63     outb %al, $LEETTORE
64     pop %rcx
65     pop %rdi
66     pop %rax
67     iret

```

